Davante

9

PROGRAMACIÓN

Manipulación de clases y objetos

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Interacción entre objetos 5

/ 3. Atributos y métodos estáticos 6

/ 4. Caso práctico 1: "Bombillas e interruptor general" 7

/ 5. Constructores y destructores 8

5.1. Constructor por defecto 8

5.2. Constructor con parámetros 9

5.3. Constructor de copia 9

5.4. Sobrecarga de constructores y destructores 10

/ 6. Modificadores de acceso 11

6.1. Modificadores de acceso para clases 11

6.2. Modificadores de acceso para miembros 12

/ 7. Métodos get/set 13

/ 8. Enumerados 14

/ 9. Bibliotecas y paquetes 15

© MEDAC ISBN: 978-84-19885-69-2
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

ÍNDICE

/ 10. Caso práctico 2: “Días de la semana” 16

/ 11. Creación e importación de bibliotecas en Netbeans 17

/ 12. Encapsulamiento de las bibliotecas 18

/ 13. Resumen y resolución del caso práctico de la unidad 19

/ 14. Bibliografía 21

OBJETIVOS

Conocer el método de paso de mensajes entre clases.

Diferenciar entre atributos convencionales y estáticos.

Utilizar métodos estáticos.

Conocer los tipos de constructores que existen.

Comprender la funcionalidad de los métodos get y set.

Conocer el concepto de visibilidad en las clases y los diferentes tipos de visibilidad.

Entender el concepto de biblioteca.

Importar clases en un proyecto.

/ 1. Introducción y contextualización práctica

Una vez explicados los conceptos de la POO y su gestión en memoria, se pasa a explicar la interacción entre objetos de distintas clases y el uso tanto de atributos como de métodos estáticos.

Se introducirá el concepto de constructores y la importancia de su uso, así como la creación de forma correcta, y se profundizará en los distintos tipos.

También se van a estudiar los métodos get y set de una clase, el porqué de su uso y su relevancia, junto al uso de los enumerados.

Por último, se introducirá la importancia del uso de bibliotecas y los paquetes en los proyectos, ya que nos permitirán importar código que ha realizado otra persona.



Fig.1. Biblioteca.

A continuación, se presenta un caso práctico a través del cual podremos aproximarnos de forma aplicada a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Organización del código"

https://on.soundcloud.com/crdoWfPWiHExWP2z7

/ 2. Interacción entre objetos

Ya hemos visto que los programas en Java están compuestos íntegramente de objetos, ya sean de clases internas, como puede ser la clase *String*, o de clases creadas por el programador.

El siguiente paso es **saber cómo los objetos interactúan entre ellos para hacer que nuestros programas funcionen correctamente**.

Los objetos se comunican entre ellos mediante sus **métodos**, con lo que podremos programar la interacción entre ellos, haciendo que sea lo compleja que necesitemos (obsérvese la figura 2).

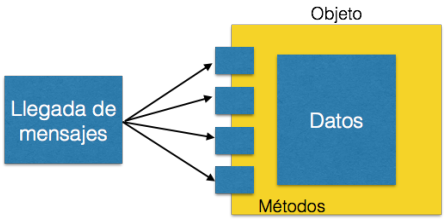


Fig.2. Métodos y mensajes de un objeto.

Cuando un objeto utiliza uno de sus métodos para comunicarse con otro, decimos que el primer objeto le envía un mensaje al segundo y que este recibe un mensaje del primero. Los mensajes también se conocen como peticiones.

En POO, definimos un **protocolo** con los mensajes que puede enviar o recibir un objeto. Dicho protocolo podrá ser modificado agregando o eliminando métodos de las clases.

Sabiendo esto, podemos decir que el comportamiento de cualquier programa que desarrollemos es:

- Creación de objetos en memoria principal RAM según se van necesitando.
- Comunicación de los mismos mediante el envío y recepción de mensajes de un objeto a otro según el programa procesa información y responde información (mostrado en la figura 3).
- Eliminación de los objetos que ya no sean necesarios para liberar espacio en la memoria RAM. El encargado de esto es el recolector de basura, no el programador ni el usuario.

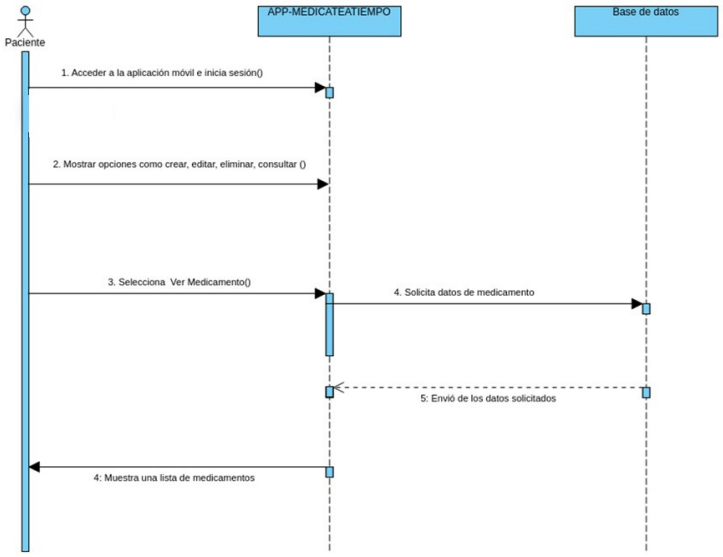


Fig.3. Envío de mensajes entre objetos.

/ 3. Atributos y métodos estáticos

Los **atributos estáticos, o atributos de clase**, son unos atributos especiales que **no dependen de los objetos, sino de las clases**. Esto quiere decir que para acceder a ellos no necesitaremos crear un objeto de la clase, sino que desde la misma clase podremos llamarlos.

Estos **atributos**, al ser de clase, son **compartidos por todos los objetos**, es decir, que si se modifican, ese cambio será visto por todos los objetos que haya creados de la clase correspondiente, o los que se creen en un futuro.

Para declarar un atributo como estático o de clase, utilizaremos la palabra reservada ***static***, la cual pondremos delante del tipo del atributo que queramos crear estático.

Un ejemplo de **atributo estático** puede ser el siguiente: Imaginemos que creamos la clase Coche (obsérvese la figura 4) con el atributo descuento común a todos los coches, ese atributo será estático.

```
// Clase Coche
class Coche {
    // Atributos de la clase
    private String marca, modelo, matricula;
    private double precio;
    // Variable estática común para todos los objetos de coche
    static double descuento = 1500;
}

public class EjemploTema5 {

    public static void main(String[] args) {
        // Creamos un objeto de tipo coche
        Coche hyundaii30 = new Coche();
        System.out.println("El descuento es: " + Coche.descuento);
    }
}
```

Fig.4. Atributo estático.

De igual forma, tenemos los **métodos estáticos**, siendo el más usado el **método *main***. Estos pertenecen a la clase, no al objeto, de ahí que también se los llame métodos de clase. La sintaxis de los métodos estáticos es la siguiente:

```
static [valor devuelto/void] nombreMetodo([parámetros])
```

Estos métodos solo podrán acceder a variables estáticas y únicamente podrán llamar a otros métodos estáticos. Al ser estos métodos de clase, no necesitaremos crear ningún objeto para poder llamarlo, ya que los métodos estáticos son métodos de clase y basta con utilizar el operador punto sobre el nombre de la clase.

¡! Sabías que...

La palabra clave *static* surgió desde el inicio del lenguaje como una característica fundamental diseñada para permitir la creación de miembros (variables y métodos) que pertenecen a la clase en sí misma, en lugar de pertenecer a instancias individuales de la clase.

```
static {
        // código
}
```

En las clases de Java tenemos lo que se conoce como un bloque estático (obsérvese la siguiente figura 5), cuya sintaxis es la siguiente:

```
public class JavaApplication10 {
    static int numero1, numero2;

    // Bloque estático
    static {
        numero1 = 10;
        numero2 = -4;
    }

    public static void main(String[] args) {
        System.out.println(numero1);
        System.out.println(numero2);
    }

}
```

Fig.5. Bloque estático.

Estos bloques se ejecutarán solo la primera vez que la clase se carga en memoria.

Vídeo 1. "Uso de métodos estáticos y de objeto"

https://bit.ly/2Ok9cz1

/ 4. Caso práctico 1: "Bombillas e interruptor general"

Planteamiento: Las variables estáticas facilitan la gestión de datos compartidos entre todas las instancias de una clase, proporcionan acceso global a ciertos valores y mejoran la eficiencia del código.

Nudo: Se quiere modelar una casa que tiene múltiples bombillas. Todas ellas disponen de un interruptor que les permite apagarse y encenderse de manera individual. Sin embargo, tenemos un interruptor general que hace que se apaguen todas o vuelvan a su estado original en el que se encuentran. Modela el comportamiento de la bombilla mediante métodos que te permitan encenderla o apagarla y te muestren el estado en el que se encuentra.

Desenlace:

```
public class Bombilla {
    public static boolean interruptorGeneral = true;
    private boolean interruptor;

    public Bombilla() {
        interruptor = false;
    }

    public void enciende() {
        interruptor = true;
    }

    public void apaga() {
        interruptor = false;
    }

    public boolean estado() {
        return interruptorGeneral && interruptor;
    }

    public String muestraEstado() {
        return estado() ? "Encendida" : "Apagada";
    }
}
```

Fig.6. Clase Bombilla.

Para la resolución de este caso, se ha creado, en primer lugar, la clase Bombilla, con los métodos que te permiten encender, apagar y comprobar el estado de las bombillas de manera individual y un interruptor general que será una variable estática (figura 6).

La siguiente (figura 7) es la clase principal que contiene el *main* donde se crean dos bombillas, una encendida y otra apagada, y se hace uso del interruptor general o variable estática (común a todos los objetos bombilla).

```
public class Main {

    public static void main(String[] args) {
        Bombilla b1, b2;
        b1 = new Bombilla();
        b2 = new Bombilla();

        b1.enciende();
        b2.apaga();

        System.out.println("b1: " + b1.muestraEstado());
        System.out.println("b2: " + b2.muestraEstado());

        Bombilla.interruptorGeneral = false;
        System.out.println("\nCortamos la luz general");
        System.out.println("b1: " + b1.muestraEstado());
        System.out.println("b2: " + b2.muestraEstado());

        Bombilla.interruptorGeneral = true;
        System.out.println("\nActivamos la luz general");
        System.out.println("b1: " + b1.muestraEstado());
        System.out.println("b2: " + b2.muestraEstado());
    }
}
```

Fig.7. Clase Principal.

/ 5. Constructores y destructores

Los **constructores** son los **encargados de inicializar los objetos de las clases**, de modo que no tiene sentido una clase sin estos métodos, ya que los objetos no se podrían crear de forma correcta.

Los constructores, sean del tipo que sean, se han de llamar exactamente igual que la clase, y, como peculiaridad, no devolverán ningún tipo de dato, ya que esta será la forma de reconocerlos para el compilador.

La forma de llamar a los constructores es mediante el **operador *new***, al que llamamos cuando vamos a instanciar un objeto.

5.1. Constructor por defecto

Si por despiste **no creamos ningún constructor en una clase**, **el compilador creará uno por defecto**, pero no inicializará los atributos de forma adecuada, solo servirá para que el proceso de compilación no falle.

La **inicialización de los atributos en un constructor por defecto** seguirá las siguientes **reglas**:

- Valor por defecto para enteros y reales: 0.
- Valor por defecto para *String*: *null*.
- Valor por defecto para *boolean*: Falso.
- Valor por defecto para objetos: null.

5.2. Constructor con parámetros

Los constructores con parámetros son otro tipo de constructores, pero, a diferencia que los constructores por defecto, estos pueden llevar **tantos parámetros como necesitemos**.

Podremos hacer tantos constructores con parámetros como combinaciones diferentes con los atributos de la clase podamos tener.

Puede pasar que pongamos el mismo nombre al parámetro que a la variable de clase, en ese caso tendremos que indicarle al constructor quién en quién.

Para indicar quién es la variable de clase utilizaremos el **operador *this***, que sirve para hacer referencia a las variables o métodos de la clase, como se puede observar en la figura 8. El operador this puede usarse igualmente en cualquier parte.

```
public Coche(String nmarca)
{
    marca = nmarca;
    modelo = "";
    matricula = "";
    precio = 0;
}

public Coche(String nmarca, String nmodelo, String nmatricula, int precio)
{
    marca = nmarca;
    modelo = nmodelo;
    this.matricula = nmatricula;
    this.precio = precio;
}
```

Fig.8. Ejemplo de constructor por defecto.

5.3. Constructor de copia

Este constructor nos ayudará a **'clonar' un objeto**, es decir, su función es la de crear un objeto nuevo a partir de otro del mismo tipo con los mismos valores en sus atributos.

El uso de este constructor viene justificado, ya que si igualamos dos objetos mediante el operador de igualdad (=), no estamos creando un objeto con los mismos atributos que al que igualamos, sino que, en realidad, estamos haciendo que el objeto que igualamos apunte a la misma dirección de memoria que el objeto al que lo igualamos.

Esto quiere decir que, cuando igualamos dos objetos del mismo tipo, al estar haciendo que apunten a la misma dirección de memoria, en realidad son el mismo objeto, y si modificamos uno de ellos, el otro 'también se modificará' por el hecho de apuntar a la misma dirección de memoria (obsérvese la siguiente figura 9).

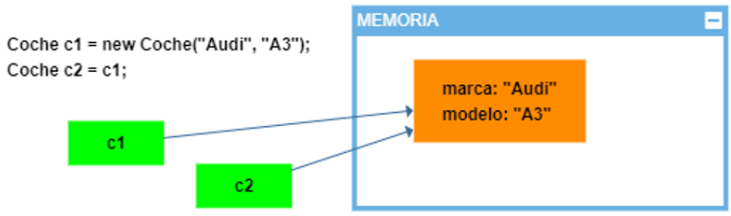


Fig.9. Igualar objetos.

Este es el problema que nos soluciona el constructor de copia, ya que, gracias a él, vamos a poder crear un objeto independiente del que copiamos, con los mismos valores en sus atributos del objeto con el que lo creamos.

En una clase, podremos tener únicamente un constructor de copia.

Un ejemplo de constructor de copia y de cómo invocarlo, lo podemos ver en la siguiente figura 10.

```
class Persona {
    private String nombre, apellidos;
    private int edad;

    public Persona(String nombre, String apellidos, int edad) {
        this.nombre = nombre;
        this.apellidos = apellidos;
        this.edad = edad;
    }

    // Constructor de copia
    public Persona( Persona copia ) {
        nombre = copia.nombre;
        apellidos = copia.apellidos;
        edad = copia.edad;
    }
}

public class EjemploTema6 {

    public static void main(String[] args) {
        Persona personal = new Persona("Pepito", "Pérez", 30);
        Persona persona2 = new Persona(personal);
        // personal y persona2 son dos objetos independientes
        // con los mismos datos
    }
}
```

Fig.10. Constructor de copia.

Audio 1. "Repaso de constructores"
https://bit.ly/304ulgy

5.4. Sobrecarga de constructores y destructores

Según las necesidades del programador, **una clase puede tener varios constructores sobrecargados**, como se muestra en la figura 11.

```
public class Persona {
    String nombre;
    String apellido;
    int edad;


    public Persona(String nombre, String apellido, int edad) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.edad = edad;
    }

    public Persona(String nombre, String apellido) {
        this.nombre = nombre;
        this.apellido = apellido;
    }

    public Persona(String nombre) {
        this.nombre = nombre;
    }
```

Fig.11. Sobrecarga de constructores.

En el primer constructor, se inicializan todos los valores; en el segundo, se inicializan nombre y apellidos, por lo que edad tomaría el valor por defecto de 0. Y por último, en el tercer constructor, solo se inicializa el nombre, por lo que apellido toma el valor null y edad, 0.

En lugar de iniciar los atributos de clase en cada constructor, es posible que un constructor invoque a otro mediante el uso del constructor genérico this(), como se puede ver en la figura 12:

```
public class Persona {
    String nombre;
    String apellido;
    int edad;


    public Persona(String nombre, String apellido, int edad) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.edad = edad;
    }

    public Persona(String nombre, String apellido) {
        this(nombre, apellido, 0);
    }

    public Persona(String nombre) {
        this(nombre, "apellidoPorDefecto");
    }
```

Fig.12. Constructor genérico this.

En este ejemplo se puede ver como el constructor con un único parámetro llama al constructor que posee dos parámetros y este a su vez llama al que tiene tres parámetros.

Por último, es necesario recordar que, a diferencia de algunos otros lenguajes de programación (como C++), **Java no tiene destructores explícitos**, sino que **cuenta con un recolector de basura** (*garbage collector*) que se encarga de liberar la memoria de los objetos que ya no son referenciados por ninguna parte del código.

Por lo tanto, los desarrolladores no necesitan preocuparse por liberar explícitamente la memoria asignada a los objetos; el *garbage collector* se encarga de esto de manera automática.

/ 6. Modificadores de acceso

Dentro del encapsulamiento y ocultación de información en las clases, podremos definir **varios niveles de protección para las clases** ubicándolos **en distintos paquetes**, los cuales podremos usar según nos convenga más en cada momento.

De la misma manera, **se puede modificar la visibilidad de los miembros de una clase**, es decir, que los atributos y métodos de una clase sean visibles para otras clases.

6.1. Modificadores de acceso para clases

Un proyecto puede tener distintos paquetes y estos a su vez están formados por un conjunto de clases, de manera que dos clases pueden definirse de la siguiente manera:

- **Clases vecinas**: Las dos clases pertenecen al mismo paquete.
- **Clases externas**: Ambas clases pertenecen a paquetes distintos.

A continuación, se muestra su esquema en la figura 13.

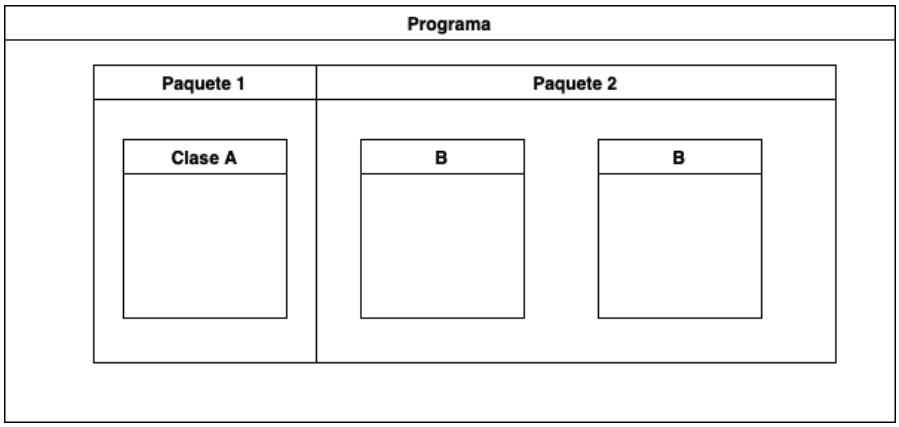


Fig.13. Representación de clases vecinas y externas.

Si partimos de la imagen con las siguientes clases:

```
class A { ... }
class B { ... }
public class C { ... }
```

Al observar la imagen la clase A, vemos que pertenece al paquete 1, mientras que las clases B y C corresponden al paquete 2, de manera que B y C son clases vecinas y son visibles por defecto entre ellas, pero no pueden acceder a la clase A, y la clase A solo podría acceder a la clase C, que es pública.

Tal y como resume la siguiente tabla 1, una clase puede verse por sus clases vecinas, y que sea visible por las clases externas dependerá de si es pública.

	Clase vecina	Clase externa
Sin modificador	Visible	No visible
Public	Visible	Visible

Tabla 1. Visibilidad entre clases.

6.2. Modificadores de acceso para miembros

De la misma manera que se modifica la visibilidad entre clases, se puede cambiar la visibilidad entre los miembros que la forman siguiendo el esquema de la figura de la tabla 2.

	Propia clase	Clase vecina	Clase externa
Private	Visible	No visible	No visible
Sin modificador	Visible	Visible	No visible
Public	Visible	Visible	Visible

Tabla 2. Visibilidad entre los miembros de una clase.

¡! Sabías que...

Estos modificadores permiten controlar la encapsulación y la visibilidad de los miembros de una clase, lo que promueve la seguridad y la coherencia en el diseño del programa.

/ 7. Métodos get/set

Ya hemos comentado que, para cumplir con los principios de encapsulación y ocultación de información, los atributos de las clases han de ser privados, pero, como hemos visto, si queremos obtener el valor de alguna de las variables, no podremos. Para solucionar esto, tenemos los métodos *getters* u observadores.

Los métodos getters van a permitir obtener el valor de los atributos que queramos, y cumplirán con los principios de encapsulación y ocultación de la información.

Normalmente, necesitaremos generar un método get para cada uno de los atributos de nuestra clase. Estos métodos no recibirán ningún parámetro y devolverán el dato que queramos observar.

Disponemos de los métodos get para poder observar u obtener el valor de una variable de clase, pero ¿y si queremos modificar el valor de dicha variable?

Hasta ahora el hecho de modificar el valor de una variable, más allá de los constructores, no era posible, por el hecho de ser estas privadas.

Para resolver este problema, aplicamos el concepto de reflexión a través de los métodos *setters* o modificadores, que serán los encargados de modificar el valor de una variable de nuestra clase, cumpliendo con los principios de encapsulación y ocultación de la información.

Normalmente, necesitaremos crear un método set para cada uno de los atributos de nuestra clase. Estos métodos recibirán el nuevo valor a asignar en el atributo, como parámetro, y no devolverán ningún dato.

En la siguiente figura 14, se pueden observar los getter/setter para los atributos de la clase persona: nombre, apellido y edad:

```java
public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public String getApellido() {
    return apellido;
}

public void setApellido(String apellido) {
    this.apellido = apellido;
}

public int getEdad() {
    return edad;
}

public void setEdad(int edad) {
    this.edad = edad;
}
```

Fig.14. Getter y setter de los atributos.



Vídeo 2. "Creación rápida de constructores, getters y setters"

https://bit.ly/309Ejm8



/ 8. Enumerados

En Java, los enumerados (también conocidos como **enums**) son tipos de datos especiales que **permiten definir un conjunto fijo de constantes con nombre**. Estas constantes son valores predefinidos que representan las posibles opciones para una variable.

Para declarar un enum en Java, utilizamos la palabra clave enum, seguida del nombre del enum y la lista de constantes entre llaves {}. Cada constante enum se separa por comas y puede llevar opcionalmente valores adicionales asociados.

```
publicenumDiaSemana {
        LUNES, MARTES, MIERCOLES, JUEVES, VIERNES,
SABADO, DOMINGO
}
```

Algoritmo 1. Definición enum.

Después de definir un enum, se pueden declarar variables utilizando el enum y asignarles valores de las constantes definidas.

```
DiaSemanadia = DiaSemana.MARTES;
```

Algoritmo 2. Declaración variable.

Los enums, en Java, pueden tener campos, métodos y constructores, al igual que las clases regulares.

```
publicenumDiaSemana {
        LUNES(1), MARTES(2), MIERCOLES(3), JUEVES(4),
VIERNES(5), SABADO(6), DOMINGO(7);
        private final int valor;

        DiaSemana(int valor) {
                this.valor = valor;
        }
        publicintgetValor() {
                return valor;
        }
}
```

Algoritmo 3. Valor del día.

Los enums son especialmente útiles con las declaraciones *switch*, ya que permiten escribir código más limpio y legible.

```
switch (dia) {
        case LUNES:
                System.out.println("Es lunes");
        break;
        case MARTES:
                System.out.println("Es martes");
        break;
        // Otros casos...
}
```

Algoritmo 4. Imprimir valor.

Los enums son útiles para definir un conjunto fijo de constantes que están relacionadas de alguna manera. Se utilizan comúnmente para mejorar la legibilidad y la mantenibilidad del código, especialmente en casos donde hay un conjunto limitado de opciones posibles.

¡! Sabías que...

Los enum surgieron con la versión 5.0 del lenguaje, lanzada en 2004. Antes de su introducción, los desarrolladores solían usar constantes enteras o cadenas de caracteres para representar conjuntos fijos de valores.

/ 9. Bibliotecas y paquetes

En la gran mayoría de lenguajes de programación que existen hoy en día, tenemos el concepto de biblioteca, o *library* en inglés.

Una biblioteca es un conjunto de clases que comparten una misma funcionalidad y pueden ser encapsuladas en un mismo paquete.

Las bibliotecas nos van a permitir llevar la reutilización de código y la ocultación de información a otro nivel, ya que, cuando utilizamos una, nos descargamos un paquete que podremos integrar en nuestro proyecto.

Un punto fuerte de las bibliotecas es que **cualquiera puede crear una y compartirla con el resto del mundo**. Por ejemplo, podemos crear una biblioteca para gestionar el acceso a una base de datos concreta, compilarla y compartirla para que otras personas puedan usarla. De esta manera resolvemos un problema concreto y, además, ayudamos a personas que pueden estar iniciándose en el mundo de la programación o simplemente a gente que necesite esa funcionalidad para sus propios proyectos.

Algunos lenguajes de programación tienen la posibilidad de poder importar bibliotecas hechas en otros lenguajes, aumentando así de una forma enorme su versatilidad.



Vídeo 3. "Crear estructura de paquetes"
https://bit.ly/2WfbraY



En nuestro caso, el lenguaje de programación **Java** nos proporciona el **comando *import***, con el que podremos importar clases de las bibliotecas que usemos, tanto propias como externas.

La sintaxis del comando *import* es la siguiente:

```
importnombrepaquete.nombreclase;
```

También podremos importar un paquete completo de la siguiente forma:

```
importnombrepaquete.*;
```

Como vemos, mediante el **comodín asterisco (*)** podremos importar todas las clases que contenga un paquete, aunque lo normal será importar solo las clases que necesitemos, ya que cuantas más clases importemos, más código tendrá nuestro proyecto y más tamaño ocupará en disco.

En un proyecto podremos importar tantas bibliotecas, propias o externas, como necesitemos. Las **bibliotecas propias** son las que trae incorporadas nuestro JDK, como puede ser la biblioteca *System*, utilizada para mostrar datos por consola.

Las **bibliotecas externas** son las que nos descargamos e integramos a nuestros proyectos pero están hechas por terceros, como puede ser una biblioteca para el acceso a una base de datos.

/ 10. Caso práctico 2: "Días de la semana"

Planteamiento: Las enumeraciones o enums, en Java, permiten definir un conjunto de constantes con nombre de manera más clara y segura que usando simples variables o literales constantes.

Nudo: Crea un programa en Java que utilice un enum para representar los días de la semana. El programa debe tener la capacidad de imprimir todos los días de la semana:

- Dado un día específico, imprimir el siguiente día de la semana.
- Dado un día específico, imprimir el día anterior de la semana.
- Comprobar si un día dado es laborable (de lunes a viernes) o no laborable (sábado y domingo).

Desenlace:

```
public enum DiaSemana {
    LUNES,
    MARTES,
    MIERCOLES,
    JUEVES,
    VIERNES,
    SABADO,
    DOMINGO;

    // Método para obtener el siguiente día de la semana
    public DiaSemana siguiente() {
        return values()[(this.ordinal() + 1) % values().length];
    }

    // Método para obtener el día anterior de la semana
    public DiaSemana anterior() {
        return values()[(this.ordinal() - 1 + values().length) % values().length];
    }

    // Método para verificar si es un día laboral
    public boolean esLaboral() {
        switch (this) {
            case SABADO:
            case DOMINGO:
                return false;
            default:
                return true;
        }
    }
}
```

Fig.15. Definición del enum.

Se ha separado el código en dos clases que comprenden la definición del enum y la clase principal.

La definición del enum comprende (figura 15):

- La definición del enum con cada uno de los días de la semana.
- El método siguiente calcula el siguiente día de la semana utilizando el índice ordinal del día actual.
- El método anterior calcula el día anterior de manera similar, asegurando que se maneje correctamente el ciclo de días.
- El método esLaboral determina si el día es laborable (de lunes a viernes) o no (sábado y domingo).

La clase principal imprime todos los días de la semana y una vez definida una variable con el día actual, imprime el siguiente día, el anterior y comprueba si es laboral (figura 16).

```
public class EjercicioEnum {
    public static void main(String[] args) {
        // Imprimir todos los días de la semana
        System.out.println("Todos los días de la semana:");
        for (DiaSemana dia : DiaSemana.values()) {
            System.out.println(dia);
        }

        // Día específico
        DiaSemana diaActual = DiaSemana.MIERCOLES;
        System.out.println("\nDía actual: " + diaActual);

        // Imprimir el siguiente día
        DiaSemana siguienteDia = diaActual.siguiente();
        System.out.println("Siguiente día: " + siguienteDia);

        // Imprimir el día anterior
        DiaSemana anteriorDia = diaActual.anterior();
        System.out.println("Día anterior: " + anteriorDia);

        // Comprobar si es un día laboral
        boolean esLaboral = diaActual.esLaboral();
        System.out.println("¿Es laboral? " + (esLaboral ? "Sí" : "No"));

    }
}
```

Fig.16. Definición de la clase principal.

/ 11. Creación e importación de bibliotecas en Netbeans

NetBeans nos ofrece la posibilidad de **crear e importar una biblioteca a partir de nuestras clases, para así poder utilizarla en futuros proyectos**.

Para ello, tenemos que crear un proyecto con ese fin. Cuando seleccionemos nuevo proyecto, elegiremos la opción *Java Class Library*, que es el tipo de proyecto para crear una biblioteca.

En la siguiente pantalla, podremos nombrar a nuestra biblioteca y elegir dónde se creará nuestro proyecto. Por el momento, todo es muy parecido a la creación de un proyecto normal y corriente.

Una vez con el proyecto vacío ya listo, crearemos tantos paquetes como nos hagan falta, y dentro de ellos crearemos las clases que queramos que contenga nuestra biblioteca.

Una vez tengamos ya montada la estructura de paquetes y clases, previamente testeadas, solo nos falta crear el fichero de biblioteca.

Los ficheros de biblioteca en Java son ficheros cuya extensión es .jar, y serán los que podamos importar más adelante.

Para crear este fichero, pulsaremos en el menú *Run* y seleccionaremos la opción *Clean and Build project*. Cuando termine de generar la biblioteca, tendremos una carpeta llamada *build* en nuestro proyecto, con el fichero .jar de nuestra biblioteca creado y listo para usarse.

En el caso de que queramos importar una biblioteca, debemos ir a la carpeta *Libraries*, donde están todas las bibliotecas que usamos en dicho proyecto, ya sean propias o externas.

Pulsaremos con el botón derecho sobre la carpeta *Libraries* y elegiremos la opción *Add JAR/Folder*, y se nos abrirá una ventana para poder elegir un fichero .jar, que será la biblioteca que vamos a usar (figura 17).

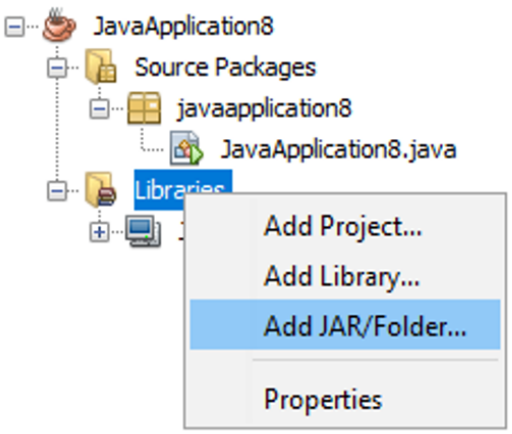


Fig.17. Agregando una biblioteca.

Una vez seleccionada la biblioteca, observamos que nos aparecerá bajo la carpeta *Libraries*, siendo esto el indicador de que hemos realizado correctamente el proceso de importación.

A partir de este momento, cada vez que compilemos nuestro proyecto, también se compilará el código de la biblioteca, de manera que podremos importar todos los elementos que posea: clases, imágenes..., mediante la sentencia *import*.

/ 12. Encapsulamiento de las bibliotecas

Como ya sabemos, las bibliotecas están compuestas de clases, y en estas tendremos atributos a los que debemos aplicar un ámbito de visibilidad obligatoriamente.

Como no podrían ser menos, **las bibliotecas también se ven afectadas por los ámbitos de visibilidad**. En este caso, podremos crear **clases privadas**, las cuales podremos utilizar en nuestra biblioteca sin ningún problema, pero no se podrán emplear fuera de ella, es más, desde fuera ni siquiera se sabrá que existen.

Esto puede ser muy útil si necesitamos de alguna clase intermedia para poder llegar a solucionar un problema, pero, al ser intermedia, no nos interesa que sea visible desde fuera.

También puede darse el caso de que nos interese declarar alguna variable de clase pública, para que pueda ser accedida desde fuera, en este caso, podremos utilizar esa variable pública de una clase de la biblioteca.

Uno de los pilares fundamentales de la programación dirigida a objetos es el **encapsulamiento**. Este nos proporciona cohesión en los datos que forman un objeto, pero, hablando de bibliotecas, ¿se siguen cumpliendo este pilar de la programación dirigida a objetos?

Cuando nos centramos en las bibliotecas, podemos ver claramente que sí cumplen con el principio de encapsulamiento, y además de este, cumplen también con el de ocultación de la información, ya que estos dos principios van de la mano.

Empecemos por el **principio de encapsulamiento**. Las bibliotecas lo cumplen, ya que todos los paquetes y clases que las forman deberán estar organizados de tal forma que sean coherentes y que cumplan una funcionalidad común.

Si las bibliotecas no estuvieran compuestas por paquetes, que a su vez contienen clases con funcionalidades con el mismo propósito, el encapsulamiento de estas sería mucho más débil, ya que estarían todas mezcladas sin ningún sentido.

En cuanto al **principio de ocultación de información**, las bibliotecas lo cumplen a la perfección, ya que todas las clases que las componen se pueden utilizar, pero no se puede ver su contenido, así que todo lo que haya sido declarado con un ámbito de visibilidad privado será totalmente opaco para quien esté utilizando dicha biblioteca.

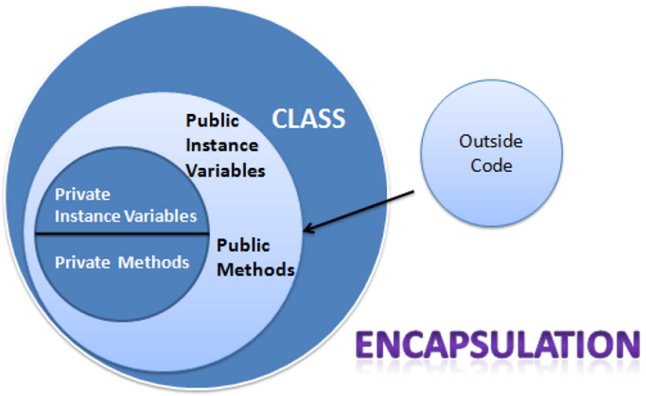


Fig.18. Ocultación de la información.

/ 13. Resumen y resolución del caso práctico de la unidad

En este tema se ha destacado que, en la programación orientada a objetos, es fundamental conocer el método de paso de mensajes entre clases, que permite la comunicación entre objetos mediante la invocación de métodos.

También se ha diferenciado entre atributos convencionales y estáticos, destacando su importancia en la estructura de clases y el uso de métodos estáticos, útiles para funciones independientes del estado del objeto.

Se ha profundizado en los tipos de constructores disponibles, esenciales para la inicialización de objetos, como el constructor por defecto, los constructores parametrizados y los constructores de copia. Y se ha explicado la funcionalidad de los métodos get y set para acceder a datos privados y modificarlos.

Además, se ha introducido el concepto de visibilidad entre clases y un nuevo tipo definido de datos: los enums.

Por último, se ha visto el concepto de biblioteca y su relevancia en la reutilización de código y se ha detallado cómo importar clases en un proyecto, facilitando la integración de funcionalidades preexistentes.

A continuación podrás ver el esquema resumen del tema:

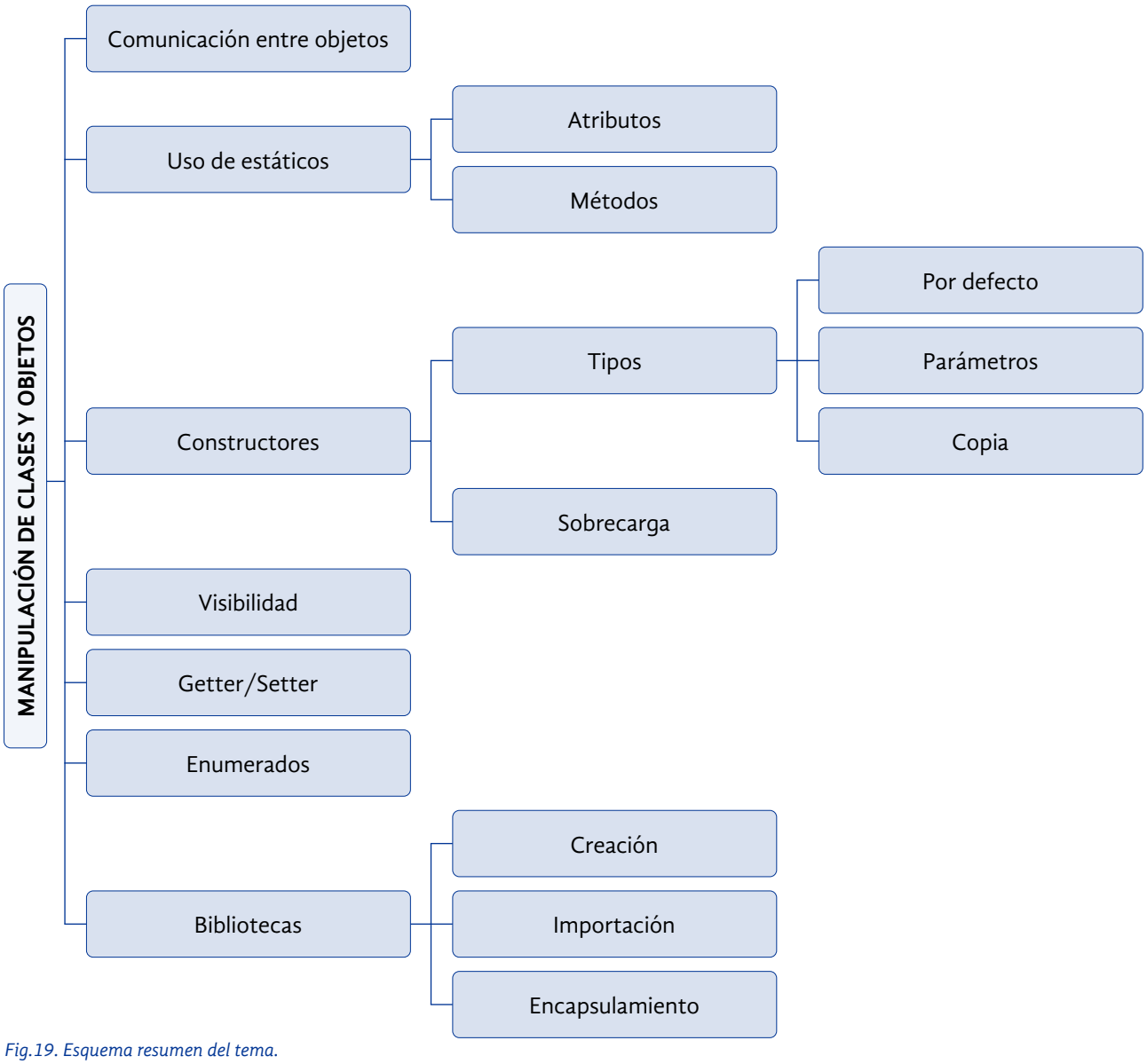


Fig.19. Esquema resumen del tema.

Resolución del caso práctico de la unidad

Siempre que en un programa tengamos muchas clases y, por lo tanto, muchos ficheros de código, lo más sensato será organizar el código para hacerlo mucho más fácil de leer, entender y, en su caso, modificar. Para ello, podemos optar por dos opciones: crear paquetes o crear bibliotecas.

La opción de crear paquetes sería la más fácil y más recomendada cuando las clases no se van a compartir con terceras personas, mientras que, si las vamos a compartir con terceros, lo más lógico sería crear una biblioteca por cada paquete que creemos, para que, así, solo tengamos que compartir el fichero creado con toda la funcionalidad ya operativa. Esto no quiere decir que no se puedan crear bibliotecas para nuestros proyectos, la idea es que no se pueda modificar el código en una biblioteca, y si necesitásemos hacerlo, tendríamos que volver a recompilarla por completo.

/ 14. Bibliografía

Bhargava, A. Y. (2019). *Algoritmos: Una guía ilustrada para programadores y curiosos*. Madrid: Ediciones Anaya Multimedia (Grupo Anaya).

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Caro, C. M., Ramos, A. N. y Barceló, A. V. (2002). *Introducción a la programación con orientación a objetos*. Prentice Hall.

Knuth, D. E. (1997). *The Art of Computer Programming* (Vol. 1: Fundamental Algorithms) (3.ª ed.). Addison-Wesley.

Manuel, P. G. J. (2022). *Entornos de desarrollo*. Ediciones Paraninfo, S.A.

Ojeda, F. C. (2001). *Introducción a la programación*. Anaya Multimedia.

Schildt, H. (2018). *Java: The complete reference* (11.ª ed.). McGraw-Hill Education.